

Procesos Java

- La clase **Runtime** representa el entorno de ejecución de una aplicación Java. Cada aplicación tiene una instancia de la clase Runtime
- *static Runtime getRuntime()*: devuelve el objeto asociado a la aplicación en curso
- Clase **Process** Representa un proceso, Siempre tiene que instanciarse.
- *Process exec(String comando)*: Ejecuta el comando en un proceso separado. (**exec** Deprecado -> **ProcessBuilder**)

```
public class Ejemplol {  
    public static void main(String[] args) {  
        Runtime r = Runtime.getRuntime();  
        String comando= "NOTEPAD";  
        Process p;  
        try {  
            p = r.exec( comando );  
        } catch (Exception e) {  
            System.out.println("Error en: "+comando );  
            e.printStackTrace();  
        }  
    }  
} //Ejemplol
```

Obtiene el entorno en el que se está ejecutando la JVM y lo guarda en r

Podemos lanzar NOTEPAD, porque el S.O. lo reconoce como notepad.exe

Process p = ps...

La clase **Process** proporciona métodos para realizar la entrada desde el proceso, obtener la salida del proceso, esperar a que el proceso se complete, comprobar el estado de salida del proceso y destruir el proceso. En la siguiente tabla se muestran los más importantes:

MÉTODOS	MISIÓN
InputStream getInputStream ()	Devuelve el flujo de entrada conectado a la salida normal del subproceso. Nos permite leer el stream de salida del subproceso, es decir, podemos leer lo que el comando que ejecutamos escribió en la consola.
int waitFor ()	Provoca que el proceso actual espere hasta que el subproceso representado por el objeto Process finalice. Devuelve 0 si ha finalizado correctamente.
InputStream getErrorStream()	Devuelve el flujo de entrada conectado a la salida de error del subproceso. Nos va a permitir poder leer los posibles errores que se produzcan al lanzar el subproceso.
OutputStream getOutputStream()	Devuelve el flujo de salida conectado a la entrada normal del subproceso. Nos va a permitir escribir en el stream de entrada del subproceso, así podemos enviar datos al subproceso que se ejecute.
void destroy ()	Elimina el subproceso.
int exitValue ()	Devuelve el valor de salida del subproceso.
boolean isAlive ()	Comprueba si el subproceso representado por Process está vivo

Por defecto el proceso que se crea (o subproceso) no tiene su propia terminal o consola. Todas las operaciones de E/S serán redirigidas al proceso padre, donde se puede acceder a ellas usando los métodos **getOutputStream()**, **getInputStream()** y **getErrorStream()**. El proceso padre utiliza estos flujos para alimentar la entrada y obtener la salida del subproceso. En algunas plataformas se pueden producir bloqueos en el subproceso debido al tamaño de búfer limitado para los flujos de entrada y salida estándar.

- *int waitFor()*: Hace que el proceso actual espere a que el subprocesso representado por Process termine
- **Ejercicio:** haz un programa que lance un proceso para ver por CMD la información IP del PC ¿Qué ocurre? ¿Qué puede estar pasando?
- Investiga cómo poder visualizar la información desde el proceso anfitrión.

```
import java.io.*;
public class Ejemplo2 {
    public static void main(String[] args) {
        Runtime r = Runtime.getRuntime();
        String comando= "CMD /C DIR" ;
        Process p=null;
        try
        {
            p = r.exec( comando );
            InputStream is = p.getInputStream();
            BufferedReader br = new BufferedReader
                (new InputStreamReader (is));
            String linea;
            while((linea = br.readLine())!=null) //lee una línea
                System.out.println(linea);
            br.close();
        }
        catch (Exception e) {
            e.printStackTrace();
        }

        // COMPROBACION DE ERROR - 0 bien - 1 mal
        int exitVal;
        try {
            exitVal = p.waitFor();
            System.out.println("Valor de Salida: " + exitVal);
        } catch (InterruptedException e) {
            e.printStackTrace();
        }
    }
}
} //Ejemplo2
```

ipconfig /all;

- A partir de JDK 1.5 añade la clase **ProcessBuilder** (java.lang). el método **start()** crea una instancia de Process con los atributos de ProcessBuilder
- **Ejercicio:** busca y comprende los atributos y métodos de proceso que gestiona ProcessBuilder
- *Process p = new ProcessBuilder("programa", "Argum1").start();* Inicia un proceso nuevo en el directorio y entorno del proceso en curso

- Ejemplos:

```
ProcessBuilder pb = new ProcessBuilder("CMD", "/C","DIR");  
Process p = pb.start();
```

```
ProcessBuilder pb = new ProcessBuilder("java", "ejercicio1", comando);  
Process p = pb.start();
```



Ejecuta java ejercicio1 y le pasa como argumento la variable comando

Algunos de los métodos porporcionados por la clase **ProcessBuilder** son los siguientes:

MÉTODOS	MISIÓN
ProcessBuilder command (String argumentos ...)	Define el programa que se quiere ejecutar indicando sus argumentos como una lista de cadenas separadas por comas.
List <String> command ()	Devuelve todos los argumentos del objeto ProcessBuilder .
Map <String, String> environment ()	Devuelve en una estructura Map las variables de entorno del objeto ProcessBuilder .
ProcessBuilder redirectError (File file)	Redirige la salida de error estándar a un fichero.
ProcessBuilder redirectInput (File file)	Establece la fuente de entrada estándar en un fichero.
ProcessBuilder redirectOutput (File file)	Redirige la salida estándar a un fichero.
File directory()	Devuelve el directorio de trabajo del objeto ProcessBuilder .
ProcessBuilder directory(File directorio)	Establece el directorio de trabajo del objeto ProcessBuilder .
Process start ()	Inicia un nuevo proceso utilizando los atributos del objeto ProcessBuilder .

```
public class LanzaNotepadPB {  
  
    public static void main(String[] args) {  
  
        // Crear el ProcessBuilder indicando el programa a ejecutar  
        ProcessBuilder pb = new ProcessBuilder(command: "notepad.exe");  
  
        Process p;  
  
        try {  
  
            // Crear y lanzar el proceso  
            p = pb.start();  
  
            System.out.println(x: "Notepad iniciado.");  
  
        } catch (IOException ex) {  
  
            Logger.getLogger(name: LanzaNotepadPB.class.getName())  
                .log(level: Level.SEVERE, msg: null, thrown: ex);  
  
        }  
    }  
}
```

- *El administrador de tareas (procesos)*

Administrador de tareas

Archivo Opciones Vista

Procesos Rendimiento Historial de aplicaciones Inicio Usuarios Detalles Servicios

Nombre	Estado	8% CPU	42% Memoria
Aplicaciones (3)			
> Administrador de tareas		1,3%	29,3 MB
> Google Chrome (12)		0,3%	568,5 MB
> Herramienta Recortes		0,5%	3,1 MB
Procesos en segundo plano (95)			
> Acrobat Update Service (32 bits)		0%	0,5 MB
> Adaptador de rendimiento inver...		0%	1,4 MB
> Andrea filters APO access servic...		0%	0,3 MB
> Antimalware Service Executable		0,1%	166,5 MB
> Aplicación de subsistema de cola		0%	3,8 MB
> Application Frame Host		0%	4,6 MB

- *El administrador de tareas (procesos)*

Administrador de tareas

Archivo Opciones Vista

Procesos Rendimiento Historial de aplicaciones Inicio Usuarios **Detalles**

Nombre	Nombre del pa...	PID	Estado
AERTSr64.exe		4136	En ejecución
ApplicationFrameHo...		7660	En ejecución
armsvc.exe		4120	En ejecución
Calculator.exe	Calculadora d...	13852	Suspendido
chrome.exe		14052	En ejecución
chrome.exe		12724	En ejecución
chrome.exe		7216	En ejecución
chrome.exe		6312	En ejecución
chrome.exe		3440	En ejecución
chrome.exe		9220	En ejecución
chrome.exe		9392	En ejecución
chrome.exe		9400	En ejecución
chrome.exe		9436	En ejecución
chrome.exe		9604	En ejecución
chrome.exe		11396	En ejecución
chrome.exe		8956	En ejecución
com.docker.service		5348	En ejecución
conhost.exe		6220	En ejecución
conhost.exe		10136	En ejecución
conhost.exe		7368	En ejecución

Administrador de tareas

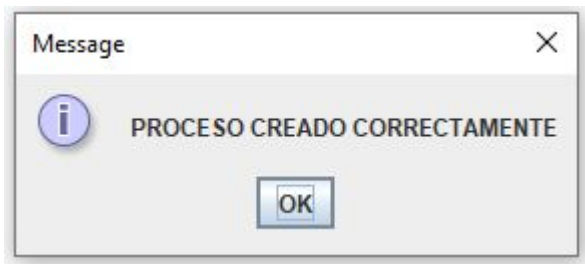
Archivo Opciones Vista

Procesos Rendimiento Historial de aplicaciones Inicio Usuarios Detalles Servicios

Nombre	Estado	8% CPU	42% Memoria
Aplicaciones (3)			
Administrador de tareas		1,3%	29,3 MB
Google Chrome (12)		0,3%	568,5 MB
Herramienta Recortes		0,5%	3,1 MB
Procesos en segundo plano (95)			
Acrobat Update Service (32 bits)		0%	0,5 MB
Adaptador de rendimiento inver...		0%	1,4 MB
Andrea filters APO access servic...		0%	0,3 MB
Antimalware Service Executable		0,1%	166,5 MB
Aplicación de subsistema de cola		0%	3,8 MB
Application Frame Host		0%	4,6 MB

EJERCICIO: Crear un programa en java con entorno gráfico tipo formulario con 3 botones, cada uno de los cuales debe abrir una aplicación:

- **Notepad:** creará un proceso nuevo de la aplicación notepad
- **App:** abrirá una aplicación java hecha por nosotros que simplemente mostrará un mensaje en ventana "Proceso creado correctamente".
- **Chrome:** abrirá la aplicación gogle Chrome:



Notas:

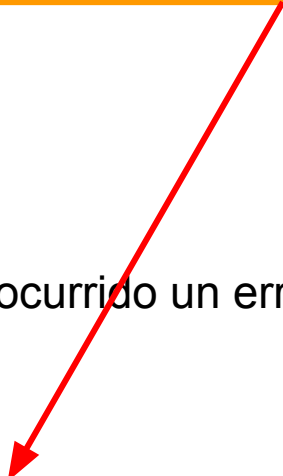
- Para la segunda opción (APP), deberás crear una clase en java que simplemente muestre el mensaje, posteriormente deberás compilar y crear el ejecutable .JAR de forma adecuada. (java -jar ,etc.)

Puede ser que necesites añadir contenido al archivo POM.xml para que el paquete jar pueda ejecutarse, [aqui](#) te explica lo que debes hacer.

- visualiza los procesos con el administrador identificando la información que consideres relevante. Observa que cada vez que se pulsa un botón, se abre un proceso nuevo (sin límite).

Opcional: piensa y realiza alguna mejora o ampliación que se te ocurra al ejercicio propuesto.

```
public void crearNuevoEditor() {  
    try {  
  
        ProcessBuilder pb = new ProcessBuilder(  
            "java",  
            "-jar",  
            "C:\\\\Users\\usuario\\Documents\\NetBeansProjects\\Editor\\Miapp.jar"  
        );  
  
        pb.start();  
  
    } catch (Exception ex) {  
        System.out.println("Ha ocurrido un error: " + ex.getMessage());  
    }  
}
```

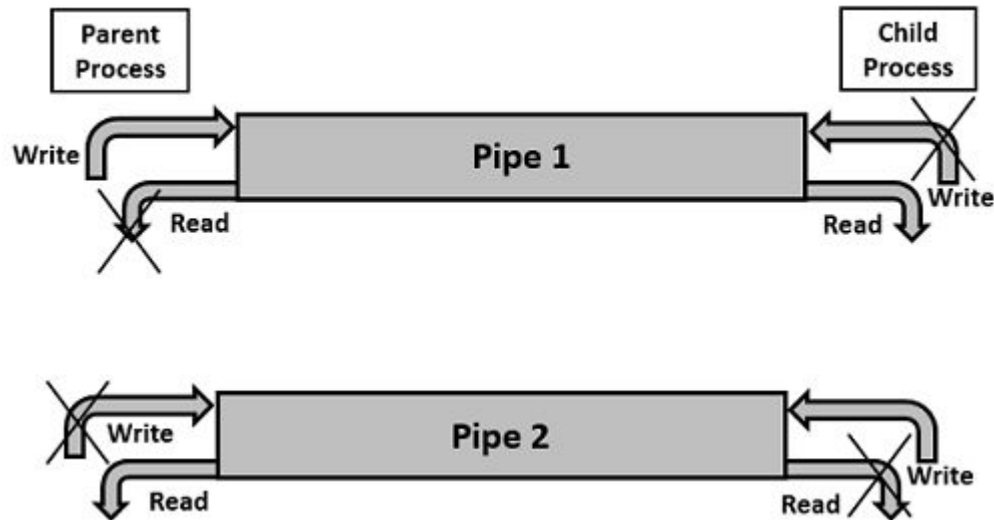


Ejecutar un .JAR desde CMD:

java -jar c: rutadelarchivo.jar

TUBERÍA:

Canal de comunicación que se establece entre dos procesos para compartir información



EJERCICIO TUBERÍA:

Ejecutar dos procesos y que uno envíe datos al otro a través de una tubería, para ello lo haremos a través de CMD usando el operador tubería " | " (**proceso 1 | proceso 2**) . Los procesos son los siguientes:

- Un proceso **Producer** que va a generar 10 caracteres aleatorios en minúscula (desde la 'a' hasta la 'z') y los enviará por la salida estándar.
- Un proceso **Consumer** que recibirá esos 10 caracteres, los pasará a mayúsculas y los sacará por pantalla en el orden inverso del que fueron recibidos. Indicando cada paso con mensajes.

Ejemplo:

```
Proceso consumer esperando datos...  
CADENA RECIBIDA: nwhqfnzrvi  
CADENA PROCESADA: IVRZNFQHWN
```

El pipe en CMD enlaza la salida por defecto del primer proceso con la entrada por defecto del segundo proceso

`system.out.println()` -> `Scanner sc new Scanner(system.in)`

Notas:

- Al igual que en el ejercicio anterior, deberás generar los .JAR ejecutables, en este caso lo ejecutaremos desde CMD y usando la tubería.
- Para generar caracteres aleatorios, pensemos en código ASCII donde cada carácter tiene asociado un número. y las opciones de Java para generar caracteres aleatorios.
- Los archivos JAR en CMD se lanzan con: `java -jar nombre.jar`

Opcional: piensa y realiza alguna mejora o ampliación que se te ocurra al ejercicio propuesto, (por ejemplo que el proceso producer reciba de vuelta el resultado por tubería).

- **Ejercicio:** Haz una programa que **reciba como argumento (en el void main)** un nombre y lo muestre por pantalla. Otro programa deberá pedir el nombre por teclado y lanzar como proceso el primer programa pasándole el nombre como parámetro.

Utiliza algo similar a esto para ejecutar el primer programa

```
ProcessBuilder pb = new ProcessBuilder("java", "ejercicio1*", comando);  
Process p = pb.start();
```

**Poner la ruta completa del programa*

Recuerda que tendrás que **capturar la salida en el programa principal**

Hacer 1º el que muestra el nombre y probar a ejecutarlo desde CMD con:
java ejercicio1 "nombre"

(Ejercicio A14)

Parámetros CMD

Parámetro	Descripción
/c	Lleva a cabo el comando especificado por <code><string></code> y a continuación, sale del procesador de comandos.
/k	Lleva a cabo el comando especificado por <code><string></code> y mantiene el procesador de comandos en ejecución.
/s	Cuando se usa con <code>/c</code> o <code>/k</code> , desencadena reglas especiales que no son de análisis que quitan las comillas primera y última (") alrededor del <code><string></code> pero dejan sin cambios el resto del comando.
/q	Desactiva el eco.
/d	Deshabilita la ejecución de comandos AutoRun.
/a	Da formato a la salida del comando como American National Standards Institute (ANSI).
/U	Da formato a la salida del comando como Unicode.
/t:{<f> <f>}	Establece los colores de fondo (<i>b</i>) y de primer plano (<i>f</i>).
/e:on	Habilita las extensiones de comando.
/e:off	Deshabilita las extensiones de comandos.
/f:on	Habilita la terminación del nombre de archivo y directorio.
/f:off	Deshabilita la terminación del nombre de archivo y directorio.
/v:on	Habilita la expansión diferida de variables de entorno.
/v:off	Deshabilita la expansión diferida de variables de entorno.
<code><string></code>	Especifica el comando que desea llevar a cabo.
/?	Muestra la ayuda en el símbolo del sistema.

- **Ejercicio:** Haz una clase en Java (Contador) que reciba como parámetros dos números enteros (delay y maximo) y cuente desde 0 hasta el máximo que le hayamos mandado con un retraso en milisegundos del delay que se le haya pasado.
 - Realiza otra clase que permita introducir esos dos parámetros y lance un proceso de la clase anterior.
 - Lanza repetidas veces el contador para visualizar como cada vez se ejecuta un contador y todos ellos funcionan de forma concurrente.

```
public static void main(String[] args){  
  
    int n1 = Integer.parseInt(args[0]);  
    int n2 = Integer.parseInt(args[1]);
```

```
java Contador.java "1000" "20"
```

Desde CMD lanza un contador cada segundo hasta el 20

Para que el contador arranque en CMD:

```
ProcessBuilder pb = new ProcessBuilder( command: "cmd.exe", command: "/K", command: "start", command: "DIR");
```

- Algunos métodos de la clase **ProcessBuilder** permiten redireccionar entradas, salidas y errores.
- En el siguiente ejemplo, un proceso ejecuta en CMD los comandos de un archivo .BAT redireccionando su entrada al mismo, mientras que la salida y los errores se redireccionan a dos ficheros independientes:

```
import java.io.File;
import java.io.IOException;
public class EjemploPB3 {
    public static void main(String args[]) throws IOException{
        ProcessBuilder pb = new ProcessBuilder("CMD");

        File fBat=new File("C:/CAPITI/PB/fichero.bat");
        File fOut=new File("c:/CAPITI/PB/salida.txt");
        File fErr=new File("c:/CAPITI/PB/error.txt");

        pb.redirectInput(fBat);
        pb.redirectOutput(fOut);
        pb.redirectError(fErr);
        pb.start();
    }
}
```

Ruta local de los archivos

Archivo .BAT:

MKDIR NUEVO

CD NUEVO

ECHO CREA FICHERO > fichero1.txt

DIR

DIRR ← Para generar error

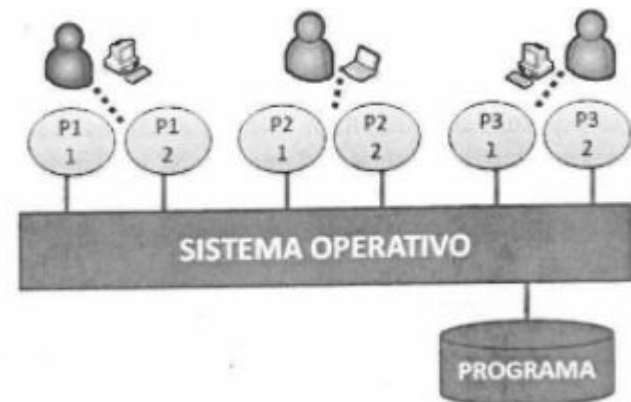
ECHO FIN COMANDOS

- **Programación Concurrente:**

- Pueden existir varios procesos activos
- La ejecución se realiza de forma intercalada por el CPU
- Un programa puede ejecutar varios procesos diferentes (internos o instancias)



3 instancias de un mismo programa
que crea un único proceso



3 instancias de un mismo programa
que crea dos procesos

- **No confundir concurrencia con paralelismo:**

- La concurrencia se da en un sistema monoprocesador
- El paralelismo se da cuando tenemos un sistema multiprocesador

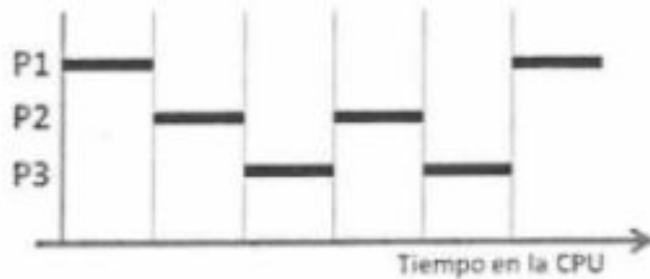


Figura 1.14. Concurrencia.

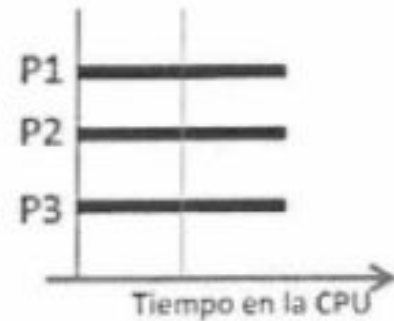


Figura 1.15. Paralelismo.

- **Programa concurrente:**

- Conjunto de acciones que se pueden ejecutar de forma simultánea

```
var1=10;  
var2=var1+5;  
var3=var2+var1;
```

Las instrucciones deben ejecutarse en ese orden concreto: **no es concurrente**

```
var1=10;  
var2=15;  
var3=readinput();
```

El orden de las instrucciones no interviene en el resultado final: **es concurrente**

- Un programa concurrente no es determinista, es decir no se puede garantizar el orden de ejecución de las instrucciones y por tanto con una misma entrada de datos podría haber distintos resultados

***Ejercicio:** Investiga y entiende las condiciones de Bernstein sobre concurrencia*

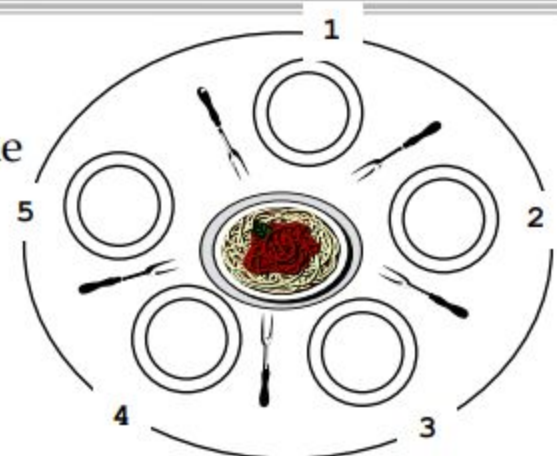
- **Acceso a los recursos**

- La concurrencia puede llevar a que varios procesos accedan de forma simultánea a un mismo recurso.
- *Ejercicio: Reflexiona sobre situaciones concretas que puedan provocar un conflicto entre procesos por el mismo recurso*
- *Es necesario implementar mecanismos de sincronización entre los procesos.*
- **Operación atómica:** conjunto de instrucciones que se ejecutan de forma indivisible (no concurrente) todas seguidas.
- **Sección crítica:** zona de código que se ejecuta de forma atómica cuando se va a acceder a un determinado recurso para que otros procesos no puedan acceder a él al mismo tiempo
- **Exclusión mutua:** Cuando un proceso está ejecutando sección crítica ningún otro proceso puede hacerlo.

- Acceso a los recursos

El problema de la cena de los filósofos

- El problema:
 - cada filósofo, alternativamente, piensa y come
 - se necesitan dos tenedores para comer
 - el de la izquierda y el de la derecha
 - la cantidad de espaguetis ¡es infinita!
 - hay que evitar que se mueran de hambre
 - por la cabezonería de los filósofos
- Salta a la vista que:
 - dos filósofos vecinos no pueden comer a la vez
 - no más de dos filósofos pueden comer a la vez
- Objetivo: desarrollar un programa concurrente que simule el sistema



Filósofo::
Mq TRUE
coge tenedores
come
deja tenedores
piensa
FMq

```

public static void main(String[] args) {
    //String nombre = args[0]; // quién escribe

    //Genera un número aleatorio para diferenciar a los proceso
    Random random = new Random();
    int numero = random.nextInt(bound: 100); // entre 0 y 99
    System.out.println("Número aleatorio: " + numero);

    //Ruta del archivo en el que va a escribir
    String archivo = "C:\\Users\\ProfDiurno\\Documents\\NetBeansProjects"
        + "\\PSP-master\\src\\main\\java\\Procesos\\recurso.txt";

    //Va a escribir un numero muy alto de entradas para que se generen problemas
    //al haber procesos concurrentes sobre el mismo recurso.
    try (FileWriter fw = new FileWriter(fileName: archivo, append: true)) {
        for (int i = 0; i < 100; i++) {
            fw.write("PROCESO: "+numero + " escribe línea " + i + "\n");
            System.out.println("PROCESO "+numero + " escribió línea " + i);
            //Thread.sleep(1);
        }
    } catch (IOException | InterruptedException e) {
    } catch (IOException e) {
        e.printStackTrace();
    }
}

```

Lanzar varios procesos al mismo tiempo y concurrentes (probar más de dos):

```

javac FileCon.java

start java FileCon.java & start java FileCon.java

```

- **Ejercicio:** Implementa dos aplicaciones:

- Una aplicación **accede a un fichero** que contiene un valor entero, lo **lee**, lo **incrementa** en 1 y lo **actualiza** en el fichero, Tiene que hacer esto 1000 veces, de forma que una vez ejecutado el fichero tiene que tener el valor 1000

```
for (i=0;i<1000;i++)
```

```
    Valor=leeValorFichero(fichero);
```

(Esquema del programa)

```
    Valor++;
```

```
    Valor=escribeValorFichero(fichero);
```

- Otra aplicación que **crea varios procesos de la anterior** de forma que todos concurrentemente utilicen el mismo fichero (Por ejemplo pedir por teclado el número de procesos a crear)
- Ejecutar y estudiar el resultado.
(Actualizarnumero y EjecutarActualizarNumero)

- **Mecanismos de sincronización**

- **Semáforos:** *Componente de bajo nivel de abstracción que permite arbitrar los **accesos a un recurso compartido** en un entorno de programación concurrente.*

*Al utilizar un semáforo, lo veremos como un **tipo dato**, que podremos instanciar. Ese **objeto semáforo** podrá tomar un determinado **conjunto de valores** y se podrá realizar con él un conjunto determinado de **operaciones**.*

*Un semáforo, tendrá también **asociada una lista de procesos suspendidos** que se encuentran a la **espera de entrar en el mismo**.*

*los **valores** que toma un semáforo son:*

Valor igual a 0. *Indica que el semáforo está **cerrado** (No quedan recursos libres por asignar).*

Valor mayor de 0. *El semáforo está **abierto** (Quedan recursos libres por asignar).*

- **Mecanismos de sincronización**

Cualquier **semáforo** permite **dos operaciones seguras**

- **objSemaforo.wait():** Si el semáforo no es nulo (está **abierto**) **decrementa en uno el valor del semáforo**. Si el valor del semáforo es nulo (está **cerrado**), **el proceso que lo ejecuta se suspende y se encola** en la lista de procesos en **espera** del semáforo.
- **objSemaforo.signal():** Si hay algún **proceso en la lista de procesos** del semáforo, **activa uno de ellos para que ejecute** la sentencia que sigue al **wait** que lo **suspendió**. **Si no** hay procesos en espera en la lista **incrementa en 1 el valor del semáforo**.

También permite realizar una **operación no segura**, que es la **inicialización del valor del semáforo**. Ese valor indicará **cuántos procesos pueden entrar concurrentemente** en él. Esta inicialización la realizaremos al crear el semáforo.

- **Mecanismos de sincronización**

- **Monitores:** *Componente de alto nivel de abstracción que gestiona recursos a los que se accede de forma concurrente*

*Los monitores encierran en su **interior los recursos o variables compartidas** como componentes **privados** y **garantizan** el acceso a ellas en **exclusión mutua**.*

*La **declaración** de un monitor incluye:*

- *Declaración de las **constantes, variables, procedimientos y funciones** que son **privados** del monitor (solo el monitor tiene visibilidad sobre ellos).*
- *Declaración de los **procedimientos y funciones** que el monitor expone (**públicos**) y que constituyen la **interfaz** a través de las que los procesos acceden al monitor.*
- ***Cuerpo** del monitor, constituido por un bloque de **código** que se ejecuta al ser instanciado o inicializado el monitor. Su finalidad es inicializar las variables y estructuras internas del monitor.*
- *Tiene asociada una **lista** en la que se incluyen los **procesos** que al tratar de acceder al monitor son **suspendidos**.*

- **Mecanismos de sincronización**

- **Mensajes:** Técnica para aportar sincronización entre procesos y permitir la exclusión mutua. No requiere de memoria compartida.

El paso de los mensajes puede ser:

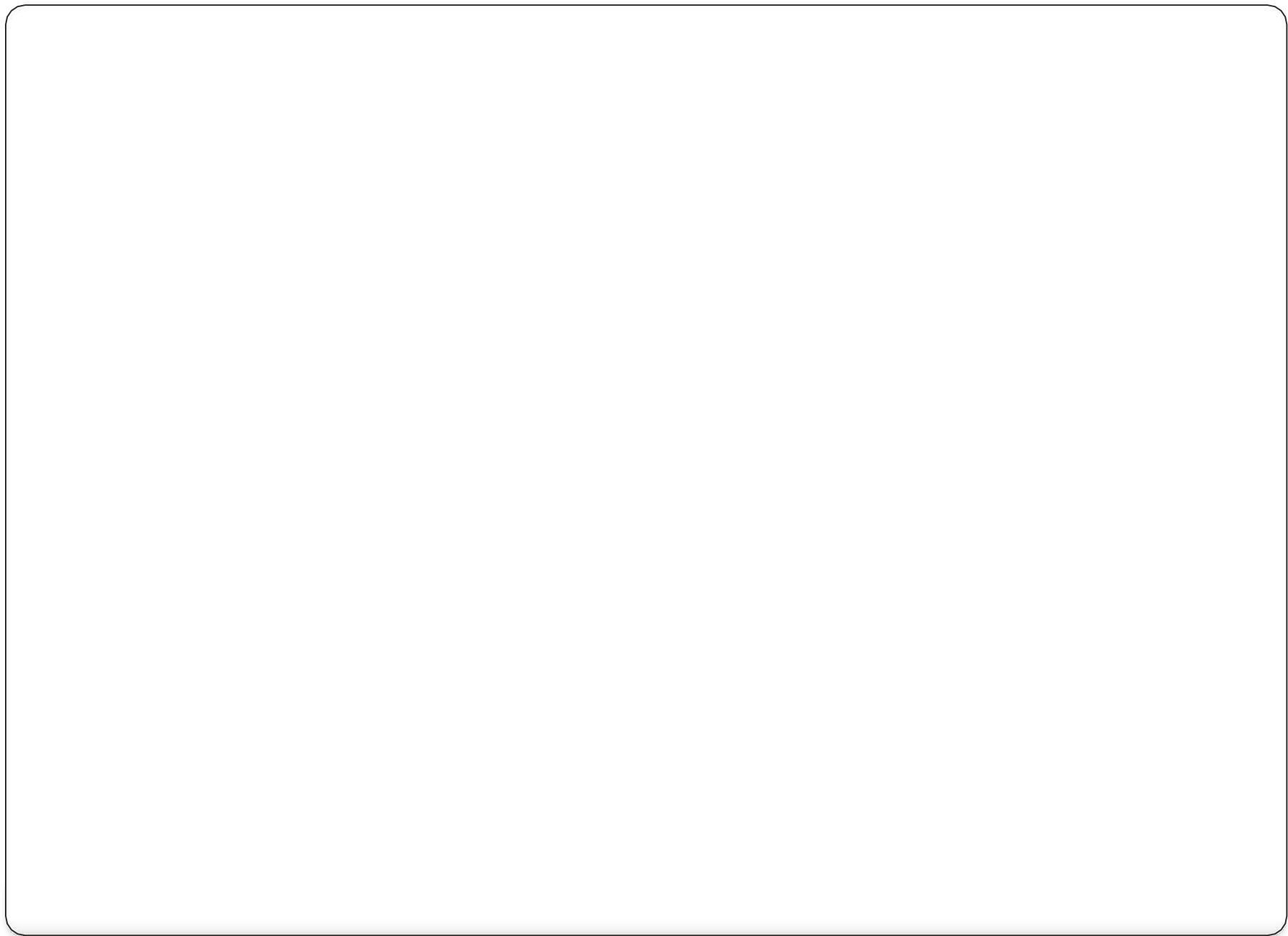
- **Asíncrono:** El que envía no espera a que el mensaje sea recibido, para ello se utilizan buzones o colas.
- **Síncrono:** El que envía espera a que se reciba (bloqueante). Por ejemplo los RPC

- **Ejercicio:** Entiende el siguiente código y pruébalo para verificar cómo funciona el acceso a un recurso sincronizado mediante **FileLock** y la definición de la **sección crítica** del código:

Código [aquí](#)

```
//*****  
//Sección crítica  
bloqueo = raf.getChannel().lock();  
//bloqueamos el canal de acceso al fichero. Obtenemos el objeto que  
//representa el bloqueo para después poder liberarlo  
System.out.println("Proceso"+ orden +  
    ": ENTRA sección");  
// Lectura del fichero  
valor = raf.readInt(); //leemos el valor  
valor++; //incrementamos  
raf.seek(pos:0); //volvemos a colocarnos al principio del fichero  
raf.writeInt(v:valor); //escribimos el valor  
System.out.println("Proceso"+ orden +  
    ": SALE sección");  
bloqueo.release(); //Liberamos el bloqueo del canal del fichero  
bloqueo = null;  
//Fin sección crítica  
//*****
```

- **Ejercicio:** Aplica el FileLocker al ejemplo anterior de escritura en el archivo para que funcione sin producir errores



- **Programa concurrente en Java: HILOS**

- Un hilo es la menor estructura de programación que se ejecuta de forma secuencial (una instrucción detrás de la otra). Es la **unidad de ejecución mínima**
- Un hilo **comparte recursos** con otros hilos de la misma tarea a diferencia de los procesos que tienen sus recursos independientes
- Los **cambios de contexto son mucho más ágiles** entre hilos, cambiar de proceso es más costoso.
- Un proceso puede tener uno o varios Hilos. Para hacer un programa concurrente, crearemos diferentes hilos de ejecución
- **Windows está basado en hilos y Linux en procesos** por lo que para usar hilos requiere de algún complemento: threads.h de POSIX pej.

- **Hilos en Java**

Java dispone de la **clase Thread** que permite ejecutar código en un hilo independiente.

- Una forma es crear una clase heredada de la clase Thread y sobrecargar el método run(). Se inicia la ejecución con start().

```
public class Hilo1 extends Thread {  
    public void run() {  
        for (int i=0;i<20;i++)  
            System.out.println("Escribiendo linea desde Hilo1");  
    }  
}
```

Dentro de **run()** va el hilo que se ejecutará de forma simultánea al resto

```
public class Principal {  
    public static void main(String[] args) {  
        Hilo1 hs=new Hilo1();  
        hs.start();  
        for (int i=0;i<20;i++)  
            System.out.println("Escribiendo linea desde principal");  
    }  
}
```

el método **start()** lanza la ejecución del Hilo

- **Hilos en Java**

Otra forma de crear hilos es implementando la **interfaz Runnable** y declarando el método `run()`. La diferencia en este caso es que hay que instanciar un **objeto Thread** de tipo hilo

```
public class Hilo1 implements Runnable {  
    public void run() {  
        for (int i=0;i<20;i++)  
            System.out.println("Escribiendo linea desde Hilo1");  
    }  
}
```

El hilo se define igual pero implementando la interfaz **Runnable**

```
public class Principal {  
    public static void main(String[] args) {  
        Hilo1 hs=new Hilo1();  
        Thread t=new Thread(hs);  
        t.start();  
        for (int i=0;i<20;i++)  
            System.out.println("Escribiendo linea desde principal");  
    }  
}
```

Para ejecutar el hilo anterior hay que **instanciar su objeto** y además uno de tipo **Thread**.

- **Ejercicio:**

Crea una clase ejecutable que ejecute varios hilos que serán contadores, es decir:

- Crear una clase heredada de thread que será el hilo
- Tendrá como atributos un valor, un número de contador y un límite
- Tendrá un constructor que permita pasarle el número de contador (1,2,3...) y el límite del contador (valor hasta el que va a contar)
- El método run() deberá incrementar el valor del contador hasta que alcance el límite, mostrando por pantalla un mensaje tal que así:
"El contador Nº X vale YY" y cuando alcance el límite: "Contador NºX FINALIZADO"
- Desde otra clase ejecutable, crear y ejecutar varios hilos contadores con diferentes límites.

- **Ejercicio:**

Modifica el ejercicio anterior para que haya 4 contadores cada uno con una tecla asignada, al pulsar dicha tecla se incrementará el contador asociado hasta que uno de ellos llegue a 50

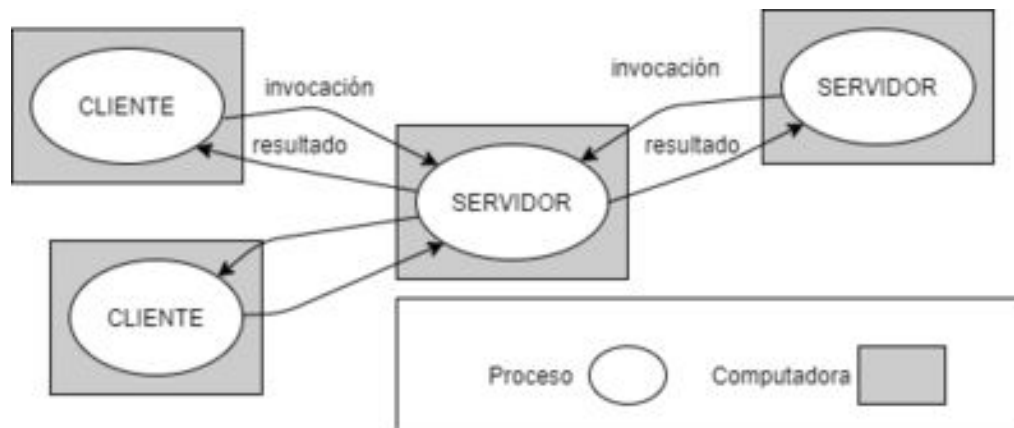
Una vez lo tengas, intenta visualizar cada contador con una barra de progreso en **JFrame** y **jProgressBar** (`jProgressBar.setValue(int)`)

- **Programación Paralela:**

- Un programa paralelo es un **tipo de programa concurrente** diseñado para ejecutarse en un sistema **multiprocesador**
- Permite que muchos **elementos de proceso** trabajen de forma **independiente** y **simultánea**
- Un sistema multiprocesador puede trabajar de dos formas:
 - **Modelo de memoria compartida:** todos acceden a la misma memoria y la comparten, los datos de uno pueden ser leídos por los demás
 - **Modelo de paso de mensajes:** cada CPU tiene su memoria independiente y para el intercambio de datos los CPUs deben comunicarse entre ellos.

- **Programación Distribuida**

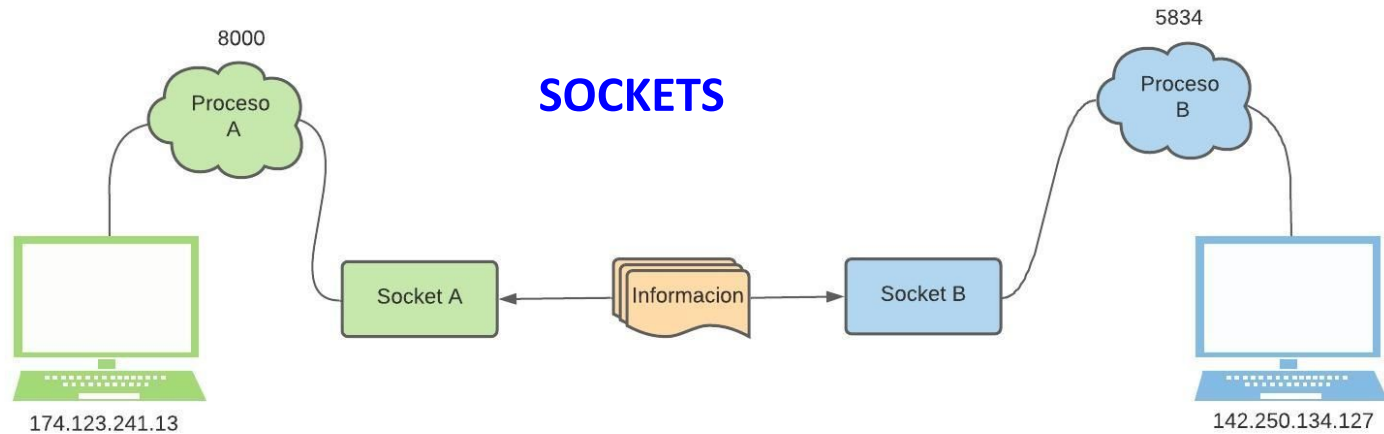
- Los componentes **Hardware** o **Software** se localizan en **diferentes computadoras** unidas entre sí mediante una **red** y se comunican y coordinan mediante **mensajes**
- **Concurrencia**: El fin es la ejecución de programas concurrentes
- **No hay reloj global**: Si requieren sincronización se hace a través de mensajes
- **Independencia de errores**: Si un sistema falla no repercute en el resto.
- **Modelo Cliente-servidor**: Es la arquitectura típica de este tipo



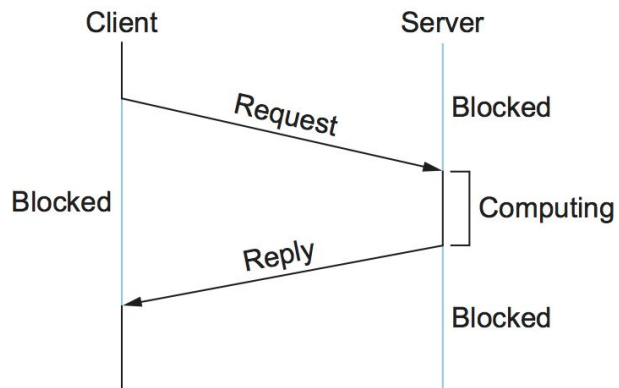
- **Programación Distribuida**

- **Tipos de comunicaciones** de la programación distribuida:
 - **Sockets:** Son los extremos en la comunicación punto a punto. Hoy en día son los más usados, se asocian a una dirección IP.
 - **RPC (Remote Procedure Call):** Llamada a procedimiento remoto, permite a un cliente ejecutar un proceso en un servidor
 - **RMI (Remote Method Invocation):** Invocación remota de objetos, un objeto de un proceso puede invocar métodos de un objeto de otro proceso diferente. Java RMI da soporte de objetos distribuidos en Java

SOCKETS



RPC



RMI

Remote Method Invocation (RMI)



- **PVM: Paralell Virtual Machine**

- Software que permite **emular un marco de programación concurrente, distribuida y de propósito general**
- Utiliza grupos de **ordenadores conectados** con sus propias arquitecturas, redes y Sistemas operativos para ser **usados como un único gran ordenador paralelo**
- Se usa en todo el mundo. **Problemas complejos** (científicos, médicos, industriales..) pueden resolverse **aprovechando la potencia y memoria** de todos los equipos conectados
- Se compone de 3 partes
 - **Demonio(PVMD3)**:reside en todos los equipos que componen la máquina virtual. Uno es maestro(inicia las tareas paralelas) y el resto esclavos
 - **Biblioteca de desarrollo (API)**: funciones para ejecutar tareas, enviar mensajes y configurar la VM
 - **Consola PVM**: interprete de comandos e interfaz con el usuario